DEERWALK INSTITUTE OF TECHNOLOGY

Tribhuvan University

Institute of Science and Technology

LANGUAGE MODELING FOR NEPALI LANGUAGE USING CHARACTER-LEVEL CNN-LSTM

A PROJECT REPORT

Submitted to

Department of Computer Science and Information Technology

DWIT College

In partial fulfillment of the requirements for the Bachelor's Degree in Computer Science and Information Technology

Submitted by

Sushil Awale

June, 2019

DWIT College

DEERWALK INSTITUTE OF TECHNOLOGY

Tribhuvan University

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by SUSHIL AWALE entitled **"LANGUAGE MODELING FOR NEPALI LANGUAGE USING CHARACTER-LEVEL CNN-LSTM"** in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology be processed for the evaluation.

……

Dr. Sunil Chaudhary

HoD, Department of Computer Science

Deerwalk Institute of Technology

DWIT College

DWIT College

DEERWALK INSTITUTE OF TECHNOLOGY

Tribhuvan University

LETTER OF APPROVAL

This is to certify that this project prepared by SUSHIL AWALE entitled **"LANGUAGE MODELING FOR NEPALI LANGUAGE USING CHARACTER-LEVEL CNN-LSTM"** in partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology has been well studied. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

…… Dr. Sunil Chaudhary HoD, Department of Computer Science DWIT College	…… Mr. Hitesh Karki Chief Academic Officer DWIT College
…….. Dr. Subarna Shakya Professor, Department of Electronics and Computer Engineering Pulchowk Campus, IOE, Tribhuvan University	…….. Mr. Ritu Raj Lamsal HoD, Department of Electronics DWIT College

ACKNOWLEDGMENT

I would like to express my gratitude to my supervisor, Dr. Sunil Chaudhary, HoD, Department of Computer Science for his valuable guidance and feedback in completing my final year project titled LANGUAGE MODELING FOR NEPALI LANGUAGE USING CHARACTER-LEVEL CNN-LSTM. His knowledge and insights into the research topic proved crucial in my completion of the project.

I would also like to give my special thanks to Mr. Birodh Rijal, Lecturer, Deerwalk Institute of Technology who guided me in the early phases of my project. Finally, I would also like to thank my family and friends who gave me encouragement to complete my project within the limited time frame.

Sushil Awale

TU Exam Roll No.: 7479/072

Date: July 1, 2019

ABSTRACT

Language modeling is an essential task in natural language processing and has wider applications in downstream tasks such as speech recognition, machine translation, spelling correction, etc. Language model architectures that use word vectors to represent the vocabulary do not capture the sub-word information (i.e. morphemes) and perform poorly in case of morphologically-rich languages such as Nepali. In this project, I apply convolution to word vectors formed by concatenation of character vectors to produce feature vectors. These feature vectors capture the sub-word information of the vocabulary and are passed into an LSTM layer through a Highway network to learn a probability distribution over a set of vocabulary. The language model built in this project, achieved a perplexity score of 378.81 i.e. in each prediction the language model is equally likely to predict 379 words as the correct one.

Keywords: *Language Modeling; Text-prediction; Character-level CNN; Long Short Term Memory*

TABLE OF CONTENTS

ACKNOWLEDGMENT iii
ABSTRACT iv
TABLE OF CONTENTS v
LIST OF FIGURES vii
LIST OF TABLES viii
LIST OF ABBREVATIONS ix
CHAPTER 1: INTRODUCTION 1
1.1 Overview 1
1.2 Background and Motivation 2
1.3 Problem Statement 2
1.4 Objective of the Project 3
1.5 Scope of the Project 3
1.6 Outline of the Report 3
CHAPTER 2: REQUIREMENT AND FEASIBILITY ANALYSIS 5
2.1 Literature Review 5
2.1.1 Neural Language Models 5
2.1.2 Character-level Models 6
2.1.2 Convolutional Embedding Models 6
2.1.4 Language Modeling for Nepali 7
2.2 Requirement Analysis 7
2.2.1 Functional Requirement 7
2.2.2 Non-Functional Requirement 8
2.3 Feasibility Analysis 9
2.3.1 Technical Feasibility 9
2.3.2 Economic Feasibility 10
2.3.3 Operational Feasibility 10
2.3.4 Schedule Feasibility 11

CHAPTER 3: METHODOLOGY 13
3.1 Data Preparation 13
3.1.1 Data Collection 13
3.1.2 Data Description 13
3.1.3 Data Preparation 14
3.1.4 Data Filtering 17
3.1.5 Filtered Data Description 18
3.2 Algorithms Studied and Implemented 19
3.2.1 Algorithms 19
3.3 System Design 25
3.3.1 Flow Diagram 25
3.3.2 Activity Diagram 26
CHAPTER 4: IMPLEMENTATION AND EVALUATION 27
4.1 Tools and Technologies Used 27
4.2 Implementation 28
4.2.1 Language Model Architecture 28
4.2.3 Hyperparameters 29
4.2.4 Description of Major Files and Classes 30
4.3 Testing 32
CHAPTER 5: CONCLUSIONS AND LIMITATIONS 33
5.1. Conclusion 33
5.2. Limitations 33
REFERENCES 34
APPENDIX I 36

LIST OF FIGURES

Figure 1: Use diagram of prediction system……..... 8

Figure 2: Network diagram to identify critical path……..... 12

Figure 3: Sample of data from the corpus…..... 14

Figure 4: Algorithm to transform morphological-tokens to orthographic tokens..... 15

Figure 5: A sample paragraph after transforming to orthographic tokens……. 17

Figure 6: Representation of a token……... 20

Figure 7: Flow diagram of the system……..... 25

Figure 8: Activity diagram of prediction next word…….. 26

Figure 9: Language model architecture for an example sentence…….……. 28

Figure 10: Language model architecture in Keras……. 36

Figure 11: User interface with an example seed input…..…… 37

Figure 12: Output result displayed by the system……..……... 37

LIST OF TABLES

Table 1: Expenditure for the project..... 10

Table 2: Activity specification with WBS..... 11

Table 3: Tag sets used in algorithm for Figure 4..... 16

Table 4: Metadata after filtering the corpus…… 18

Table 5: Hyperparameters used in the training process…….. 29

LIST OF ABBREVATIONS

AJAX	Asynchronous JavaScript and XML
CASE	Computer-aided
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CPM	Critical Path Method
CSS	Cascading Style Sheet
DDR	Double Data Rate
ELRA	European Language Resource Agency
GB	Giga Bytes
Ghz	Giga Hertz
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
LSTM	Long Short Term Memory
LTS	Long Term Support
NLM	Neural Language Model
NLP	Natural Language Processing

LIST OF ABBREVATIONS

Nrs	Nepalese Rupees
OS	Operating System
POS	Part Of Speech
RAM	Random Access Memory
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
URL	Uniform Resource Locator
WBS	Work Breakdown Structure
XML	eXtended Markup Language

CHAPTER 1: INTRODUCTION

1.1 Overview

Language Modeling is one of the widely researched topic in NLP. Language Modeling can be defined as the task of calculating the probability distribution over a sequence of strings (words/characters). A language model is used to predict the next word or character given a sequence of words or characters. Traditionally, the probability distribution over a set of words were calculated on the basis of frequency of words. In a similar fashion, n-gram probability models were popular in the past. However, these frequency-based models performed poorly to predict rare words and also failed to address out-of-vocabulary words.

NLM proposed in 2003 by Bengio et al. [1] addressed these problems. The three layer model in NLM quickly became the standard language model and most of the state-of-the-art language models are a variant of the NLM.

In recent years, researchers have turned to deep learning techniques to tackle the problem of language modeling. In particular, the paper on Word2Vec by Mikolov et al. in 2013 [2] was very instrumental in the rise of using deep learning models and a number of deep learning architectures have been proposed, each producing state-of-the-art results.

The use of CNN for text has also been increasing in the past few years. CNN have the advantage of faster computing compared to RNN or LSTM which were widely used in NLP. In 2015, Kim

et al. [3] proposed a CNN-LSTM language model which outperformed the state-of-the-art language models for morphologically rich languages such as German, Hindi, Arabic, Russian, etc.

Although, wide research is being carried out in language modeling for many languages, to best of my knowledge, no published work exists for Nepali language.

1.2 Background and Motivation

Language modeling is the first task carried out in Natural Understanding. A well-built language model has been proven to be useful and it improves the results for many downstream tasks such as Machine Translation [4], Part-of-Speech tagging [5], Speech Recognition [6] etc. Hence, there is a need of research in Language Modeling for Nepali language.

Although many language modeling architecture exists for other languages, they do not tend to work for every language. For example, a language model architecture using Word2Vec as the input layer does not tend to capture the sub word information of a token (i.e. morphemes) [3]. However, in rich morphological languages such as Nepali, morphemes play a crucial role in the language. Hence, alternative language model architectures that capture the sub word information must be researched on.

1.3 Problem Statement

Language model architectures that use word vectors such as Word2Vec do not capture morphological features of language. As a result, the language models underperform for morphologically rich languages such as Nepali. A new language model architecture that captures these morphological features which are a key part of the language is required. In recent years, CNN for text has been widely used as research upon and has shown some really good results on

morphologically rich languages. Kim et al. in 2015 [3] showed that the character-level CNN as the input layer in the language model out-performed other word vector-based language models significantly. However, for Nepali language there exists no published research.

1.4 Objective of the Project

The objective of this project is:

- To build a language model for Nepali language that predicts the correct word given a sequence of previous words

1.5 Scope of the Project

The project consists of building a working language model for Nepali language. The built language model will be deployed in a web-based system with a GUI interface. The system will generate and display an orthographic token based on the previous three orthographic tokens (seed input) provided by the user. In addition to that, the system will also display top five orthographic tokens ranked by probability score.

1.6 Outline of the Report

The report is organized as follows:

Preliminary Section: This section consists of the title page, abstract, table of contents and list of figures and tables.

Introduction Section: In this section, the background of the project, problem statement, its objectives and scope are discussed.

Requirement and Feasibility Analysis Section: Literature review, Requirement analysis, and feasibility analysis make the bulk of this section.

System Design Section: The section consists of the methodology that was implemented in the project and the system design as well.

Implementation and Testing Section: The section consists of the methodology that was implemented in the project and the system design as well.

Conclusion and Recommendation: The section consists of the final findings and the recommendations that can be worked on in order to improve the project.

CHAPTER 2: REQUIREMENT AND FEASIBILITY ANALYSIS

2.1 Literature Review

Language modeling is popular problem in the field of Artificial Intelligence and NLP. A well-built language model can be used to further improve downstream tasks in NLP such as Speech Recognition [6], Machine Translation [4], etc. Formally, language modeling is defined as a probability distribution over a sequence of strings (words/characters). Traditional methods usually involve making an n-th order Markov assumption and estimating n-gram probabilities via counting and subsequent smoothing. [8] These frequency-based models are easy to train, but do not perform well for rare-words or unseen vocabulary.

2.1.1 Neural Language Models

Bengio et al. [1] proposed a NLM that addressed the n-gram data sparsity through parameterization of words as vectors (word embeddings). The NLM used the word vectors as input to the neural network which had a single hidden layer feed-forward neural network that predicted the next word in a sequence.

Bengio et al.'s architecture [1] which consisted of three layers is widely used by many of the neural language model architecture these days. The three layers are:

1. **Embedding Layer:** a layer that generates word embeddings by multiplying an index vector with a word embedding matrix

2. **Intermediate Layer/s:** one or more layers that produce an intermediate representation of the input
3. **Softmax Layer:** the final layer that produces a probability distribution over words in vocabulary set V.

Although NLMs have shown to outperform frequency-based n-gram language models [9], they do not capture sub word information such as morphemes [7].

2.1.2 Character-level Models

Character-level NLMs work around the blindness to sub-word information by taking characters as input and producing characters as output [3]. These models do not require any form of morphological tagging or manual feature engineering, and also are capable of producing new and unseen words. However, word-level models seem to outperform these models [8].

2.1.2 Convolutional Embedding Models

In recent years, many researchers have explored using character-level inputs to build word embeddings for various NLP tasks such as part-of-speech tagging [5], machine translation [4], and language modeling [3]. The convolutional models seem to capture the extra morphological information present in the tokens. [3]

Kim et al. [3], used a convolutional embedding model for language modeling which out-performed the then state-of-the-art language models for morphologically rich languages such as German, Hindi, Russian, Arabic, etc. The model processed word characters using a one-dimensional CNN with max-pooling across the sequence for each convolutional feature. The output of the CNN is

then run through two-layer Highway network [14] and finally passed to a two-layer LSTM to calculate the probability distribution using a Softmax layer.

2.1.4 Language Modeling for Nepali

Although a lot of research works have been carried out in language modeling for various languages, to best of my knowledge, no work has been published for Nepali language.

Performance of a language model depends on many factors such as vocabulary size, language characteristic, size and position of prediction window, block of text to be predicted, search strategy, prediction method, etc. Language modeling for Nepali language poses similar challenges.

Therefore, in this work, a NLM has been developed for Nepali language using character-level CNN and LSTM.

2.2 Requirement Analysis

2.2.1 Functional Requirement

The functional requirements of this project are as follows:

- The user shall enter at least three orthographic tokens in Devanagari script using the user interface.
- The system shall predict the next orthographic token and display it to the user.

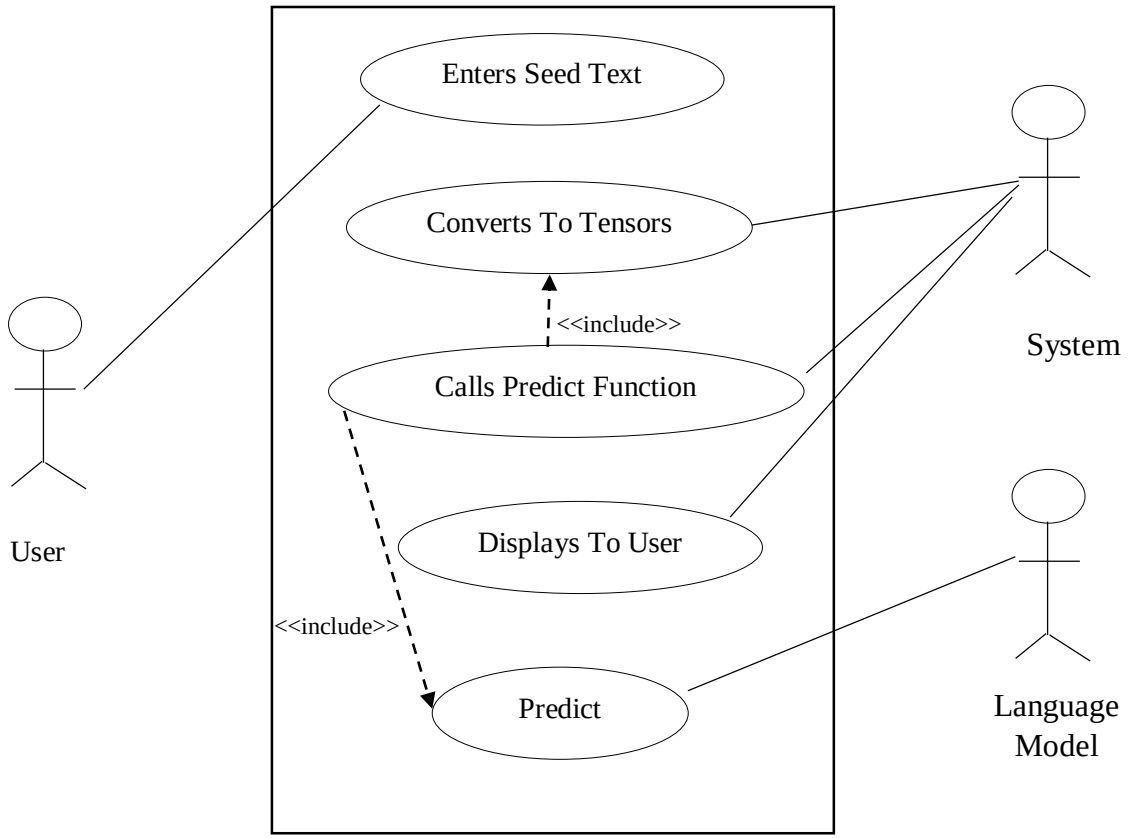

Figure 1 - Use case diagram of prediction system

Figure 1 shows the use case diagram of the prediction system developed in this project. The user performs the solitary action of entering the seed input. Then all other functions **Convert To Tensors**, **Call Predict Function** and **Display To User** are done by the system. During the call **Predict Function** the system calls the **Predict** function which is performed by the language model.

2.2.2 Non-Functional Requirement

The non-functional requirements of the project are as follows:

- The system must display the top five orthographic tokens based on probability score.
- The system response time must be less than 0.1 seconds.
- The system must have easy to use user interface.

2.3 Feasibility Analysis

After gathering of the required resources, whether the completion of the project with the gathered resource is feasible is checked using the following feasibility analysis.

2.3.1 Technical Feasibility

In this project, the system was implemented as a web-based application and it runs in systems with MacOS, Windows or Linux operating systems. The computer system must include a web browser to access the system. The web-based user interface was built using Flask, a micro framework in Python.

The primary programming language selected to build the system was Python (version 3.6.5), which is an open-source programming language. The programming language was selected due its ease of use and my experience in working with the programming language.

Keras framework was used in building the language model architecture. Keras is a high-level machine learning library built on top of Tensorflow library which allows its users to build a machine learning architecture quickly. Keras was chosen considering the limited time frame for completion of the project.

To train the language model, Google's free Google Colaboratory service was used. Google Colaboratory is a free Jupyter notebook environment that requires no setup and runs entirely in the cloud. Google Colaboratory was used to speed up the training of the language model with the free GPU service provided by it.

The tools, systems, modules and libraries needed to build the system are all open source, freely available and are easy to use. Hence, the project was determined technically feasible.

2.3.2 Economic Feasibility

The expenses incurred in the project are mostly indirect expenses. We used our personal devices to build the system and used personal internet subscription to do the research online. Moreover, training the language model, which is computationally expensive and time consuming, was done using Google's free Google Colaboratory service. However, some direct expenses were incurred in the acquisition of the data and printing the documentation. The breakdown of the expenses is given in table.

Table 1 – Expenditure for the project

S.N.	Item	Price
1	Mailing the end-user agreement to ELRA	Nrs. 300
2	Printing and Binding	Nrs. 1000
	Total	Nrs. 1300

Since this is an academic project, the incentive to complete the project was university credit score (6 credit hours). Hence, considering the costs and benefits of completing the project, the project is deemed economically feasible.

2.3.3 Operational Feasibility

The system built in the project follows a client-server architecture. The web-based application can operate in a system having Windows, Linux or MacOS. The user-interface can be accessed remotely or from within the same system where the application is hosted. The resource-intensive part of the project was training the language model. However, it is only a one-time process and

hence need not be carried out when using the system. Considering the above cases, the project is deemed operationally feasible.

2.3.4 Schedule Feasibility

The schedule feasibility analysis is carried out using the CPM method. With CPM, critical tasks were identified and interrelationship between tasks were identified which helped in planning that defines critical and non-critical tasks with the goal of preventing time-frame problems and process bottlenecks. The CPM analysis was carried out as follows:

First, the activity specification table with WBS is constructed as shown in the table:

Table 2 - Activity specification with WBS

Activity	Time (Days)	Predecessor
Data collection and Preprocessing (A)	60	-
Research on previous work and algorithms to be implemented (B)	30	-
Building the language model (C)	30	A, B
User Interface Design (D)	5	C
System Testing (E)	1	C
Documentation (F)	30	A, B, C, E

Then, identification of critical path and estimates for each activity are analyzed with the help of a network diagram, which is based on Table 2. The network diagram is shown in figure 1.

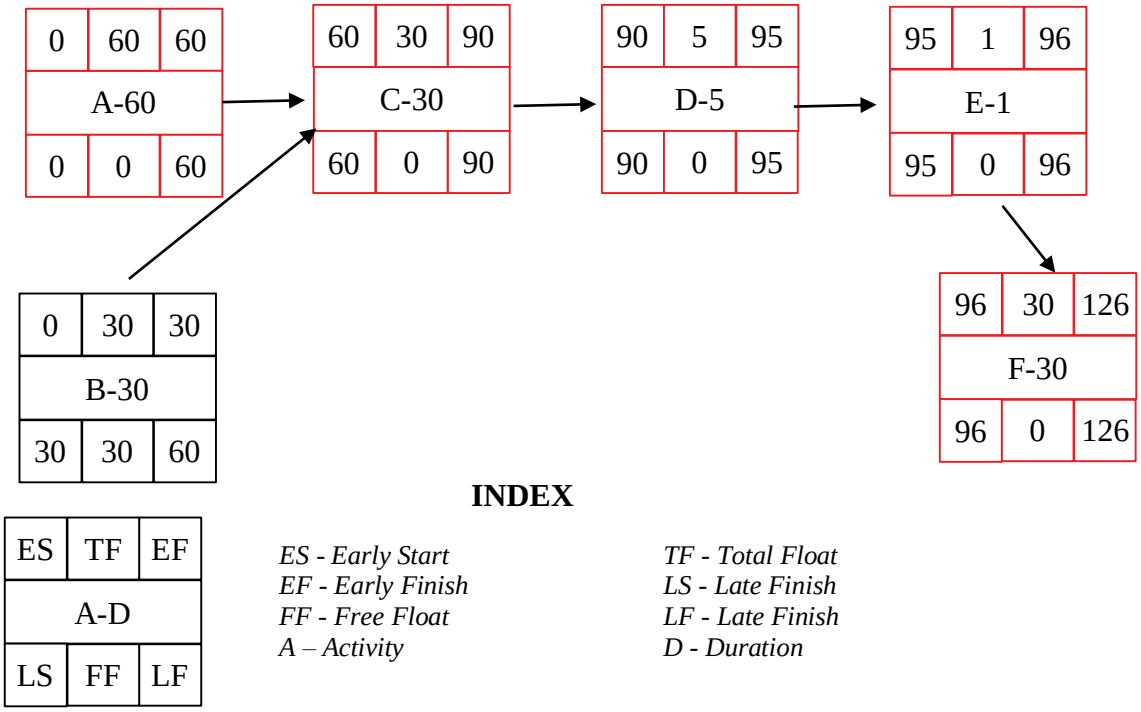

Figure 2 - Network diagram to identify critical path

Figure 2 shows the Activity Network Diagram of the project. As shown in the above figure, it is observed that the critical tasks are (A) data collection and preprocessing, (C) building the language model, (D) user-interface design, (E) system testing and (F) documentation. The total duration of the critical path (A-C-D-E-F) is 126 days, which is in the project deadline range. Hence, this project is feasible in terms of the schedule since the project is completed in time if the critical tasks are carried out within the specified task's duration in Table 2.

CHAPTER 3: METHODOLOGY

3.1 Data Preparation

3.1.1 Data Collection

Nepali Monolingual written corpus was used for training the language model. Nepali Monolingual written corpus was collected from ELRA. [9]

First, an order request form was sent to ELRA through email upon which ELRA sent a language resources end-user agreement. The contract was then printed and signed by my supervisor and it was mailed to ELRA. ELRA, then, provided a URL to download the dataset.

3.1.2 Data Description

The Nepali Monolingual written corpus consists of two corpus: core and general. The core corpus represents the collection of Nepali written texts from 15 different genres 2000 words each published between 1990 and 1992 and the general corpus consists of written texts collected from a wide range of sources such as the internet webs, newspapers, books, publishers and authors.

The written corpus is morphologically-annotated i.e. the text is divided into tokens. In this corpus, the tokens are appropriately-sized units for morphosyntactic analysis rather than orthographic tokens.

The written corpus is in XML format where a paragraph from the text is enclosed with <p> tag, the sentences in the paragraph are enclosed with <s> tag, and the words in the sentences are

enclosed with <w> tag with its POS tag specified as the value of attribute ‘ctag’. There are a total of 112 POS tags in the corpus denoted by roman alphabetic symbols and an index is maintained for the symbols.

```
<s n="2">
    <w ctag="DDX">यस</w>
    <w ctag="NN">अवधि</w>
    <w ctag="II">मा</w>
    <w ctag="DKX">कसै</w>
    <w ctag="IE">ले</w>
    <w ctag="TT">पनि</w>
    <w ctag="DKX">कुनै</w>
    <w ctag="JX">मह338वपूर्ण</w>
    <w ctag="NN">अवार्ड</w>
    <w ctag="VI">जित्न</w>
    <w ctag="VE">सके</w>
    <w ctag="IKM">को</w>
    <w ctag="VVYN1">थिएन</w>
    <w ctag="YF">।</w>
</s>
```

Figure 3 - Sample of data from the corpus

The core sample consists of 802,000 tokens and the general corpus consists of 1,400,000 tokens.

3.1.3 Data Preparation

A language model may be trained to predict the next word, next character or next sentence. Since, the objective of the project is to train a language model to predict the next orthographic token, the morphologically-tokenized tokens was concatenated to form grammatically correct orthographic tokens. The transformation task was automated by designing an algorithm based on [10] and several Nepali grammar books. [11, 12] The algorithm is shown in figure. Utilizing the XML

structure of the data, the words were then concatenated to form sentences and sentences concatenated to form paragraphs i.e. the dataset was tokenized into paragraphs and enclosed with <p> tags and given a unique id to each paragraph. All the paragraphs from different files were written in a single large XML file. A sample paragraph from the output is shown in figure. A total of 88,715 paragraphs were formed.

```
for paragraph in paragraphs
    for sentence in paragraph
       for token in sentence
         tag = getTag(token)
         if tag not in IGNORE
            if tag in POSTPOSITIONS or CLASSIFIERS
                x += token
            else
                x += space + token
    y = encodeInXML(x)
    writeToFile(y)
```

Figure 4 - Algorithm to transform morphological-tokens to orthographic tokens

Figure 4 shows the algorithm used to transform morphological tokens to orthographic tokens. The POS tag associated with the tokens were used to decide whether to ignore the tokes and whether to concatenate the tokens or not. The list of tags belonging to variable **IGNORE**, **POSTPOSITIONS** and **CLASSIFIERS** are shown in Table 3. The information is extracted from [10]

Table 3 – Tag sets used for algorithm in Figure 4

LABELS	CATEGORY	EXAMPLES	TAG
POSTPOSITIONS	Postposition	बाट, मा, माथि	||
	Plural-collective postposition	हरू	IH
	Ergative-instrumental postposition	ले	IE
	Accusative-dative postposition	लाई	IA
	Masculine genitive postposition	को	IKM
	Feminine genitive postposition	की	IKF
	Other-agreement genitive postposition	का	IKO
CLASSIFIERS	Masculine numeral classifier	#टो	MLM
	Feminine numeral classifier	(#)वटी, #टी, #ओटी	MLF
	Other-agreement numeral classifier	#वटा, #टा, #ओटा	MLO
	Unmarked numeral classifier	(#)जना	MLX
IGNORE	Foreign word		FS
	Mathematical formula		FO
	Letter of the alphabet		FZ
	Unclassifiable		FU
	NULL Tag		NULL

सो कार्यपत्रप्रति टिप्पणी गर्ने क्रममा वरिष्ठ हास्यव्यङ्ग्यकार तथा निर्देशक रामशेखर नकर्मीले प्रहसनले समाजलाई पीडाबीच पनि हँसाउँदै विसङ्गतिविरूद्ध स8249र्ष गर्न प्रेरणा दिने तथ्यमा प्रकाश पार्नुभयो । ' नेपालभाषामा हास्यव्यङ्ग्य विधा ' विषयमा कार्यपत्र प्रस्तुत गर्दै त्रिभुवन विश्वविद्यालय नेपालभाषा शिक्षण समितिका अध्यक्ष प्रा . नर्मदेश्वर प्रधानले प्रजातन्त्र , मानव अधिकारप्रतिको सचेतताका कारण हास्यव्यङ्ग्यप्रतिको अभिरूचि तथा आस्था जनमानसमा झन् झन् बलियो हुँदै आएको बताउनुभयो ।

Figure 5 - A sample paragraph after running through the algorithm

3.1.4 Data Filtering

After transforming the data into suitable format for training, the data was filtered. The following filtering measures were taken.

3.1.4.1 Removing Non-devanagari Tokens

All the tokens present in the data that were not in Devanagari tokens were removed using regular expressions. In order to detect the non-devanagari tokens using regular expressions, Unicode code points were used. Figure presents the characters found in Devanagari script and their respective Unicode code points. The tokens in which all the characters did not belong to the Devanagari code range were labelled non-devanagari tokens and removed from the data.

3.1.4.2 Replacing Tokens

The corpus consisted of tokens with font conversion errors. For example, in figure the token `स8249र्ष` consists of roman numerals in-between the Devanagari characters `स` and `र्`. These errors occur when glyphs are transformed into fonts. In the above example, the number 8249 is actually the code for the Devanagari characters `ङ्घ`.

In order to replace tokens with font conversion errors, a dictionary of tokens with font conversion errors was created by traversing through all the tokens. An XML file was used to create this dictionary, where the token was stored along with the paragraph id from where the token was extracted from. The dictionary comprised of 318 tokens. Then, the replacements for these tokens were added to the dictionary manually. The replacements were selected on the basis of context by reading the corresponding paragraphs (using paragraph id).

3.1.4.3 Removing Paragraphs

Paragraphs with less than 35 tokens were removed in order to match the sequence length for training the language model. The step was taken to only include coherent paragraphs to train the language model.

3.1.5 Filtered Data Description

After filtering the data, the following statistics were observed on the filtered data.

Table 4 – Metadata after filtering the corpus

Character count	89
Total orthographic tokens	3758521
Total unique orthographic tokens	359633
Total paragraphs	41247
Max word length	50

3.2 Algorithms Studied and Implemented

3.2.1 Algorithms

The following algorithms were used to train the language model.

3.2.1.1 Character-level Convolutional Neural Network

Character-level CNN was first introduced by Kim et. al in 2016 in a paper titled Character-Aware Neural Language Model [3]. Character-level CNN is based on CNN first introduced by LeCunn et. al. in 1989 in a paper titled Object Recognition with Gradient-based Learning. [13] CNNs have achieved state-of-the-art results on computer vision, however, in recent years they have also been shown to be effective for various NLP tasks.

CNNs typically assume the input to be an image i.e. a matrix of height h and width w with each entry in the matrix representing the pixel intensity of the image at that coordinate. Then convolution operation is applied to the matrix to generate an output matrix of lower dimension. The convolution operation is defined as the element-wise multiplication of the input matrix and a selected filter matrix. The generated output matrix is not only of a lower dimension but it also extracts certain features from the input matrix (the image).

CNNs comprises of multiple layers of convolution with nonlinear activation functions such as ReLU or tanh applied to the results. Each layer applies different filters and combines the results. A CNN, during the training phase, automatically learns the values of its filters based on the desired task.

In the case of NLP, the inputs are words or sentences, instead of image pixels. The words/sentences are represented using a matrix where each row corresponds to one word or one character. The row

is typically a vector. Normally, a one-hot vector is chosen but it can be pre-trained vectors such as word2vec, etc.

For character-level CNN, let C be the vocabulary of characters and d be the dimensionality of character embedding. Then $Q \varepsilon \mathbb{R}^{d \times |C|}$ is the matrix of character embeddings. Then a token $k \varepsilon V$ which is made up of a sequence of characters $[c_1, c_2, \ldots, c_n]$, where n is the length of the token k is represented by a matrix $C_k \varepsilon \mathbb{R}^{d \times n}$. Here, the jth column corresponds to the character vector for cj.

For example, a token represented as

		...		
		...		
		...		
		...		

Figure 6 - Representation of a token

In figure 6, each column corresponds to a character in a token and each column is a vector representation of a character. The vectors are then concatenated to for a matrix of dimensions $d \times n$, where d is the dimensionality of the character vector and n is the maximum length of the tokens. For tokens with length $< n$, zero-padding is carried out and the start and end of the tokens are dentoed by '{' and '}' respectively.

Then, a convolution between C_k and a filter $H \in \mathbb{R}^{d \times w}$ of width w is applied. Then a bias and a nonlinearity is applied to obtain a feature map $f^k \in \mathbb{R}^{l-w+1}$. Here, the *i-th* element of f^k is given by:

$$f^k[i] = \tanh((C^k[*, i{:}\, i + w - 1], H) + b) \ldots\ldots\ldots\ldots\ldots\ldots (1)$$

where,

$C^k[*, i: i + w - 1]$ is the *i-to-(i+w-1)-th* column of $C^k$and *(A,B) = Tr(AB ^{T})* is the Forbenius inner product. Finally, the max-over-time

$$y^k = max f^k[i] \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (2)$$

as the feature corresponding to the filter **H.**

Here the character-level CNN uses multiple filters of varying widths to obtain the feature vector for *k*.

$t = \sigma(W_{TY} + b_T)$ is called the transform gate,

$(1 - t)$ is called the carry gate, and

y is the output of character-level CNN

3.2.1.2 Highway Network

The highway network was first proposed by Srivastava et. al. (2015). [14] It performs the following operation

$$\mathrm{Z} = t \odot g\,(W_{HY} + b_H) + (1 - t) \odot y \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (3)$$

where,

g is a nonlinearity,

$t = \sigma(W_{TY} + b_T)$ is called the transform gate,

$(1 - t)$ is called the carry gate, and

y is the output of character-level CNN

3.2.1.3 Recurrent Neural Network

A RNN is a type of neural network architecture particularly suited for modeling sequential phenomena. At each time step *t*, an RNN takes the input vector $x_t \in \mathbb{R}^n$ and hidden state vector $h_{t-1} \in \mathbb{R}^m$ and produces the next hidden state h_t by applying the following recursive operation:

$$h_t = f(W_{x_t} + Uh_{t-1} + b) \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (4)$$

Here, $W \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$ are parameters of an affine transformation and f is an element-wise non-linearity. In theory, RNN can summarize all historical information up to time t with the hidden state h_t. In practice, however, learning long-range dependencies with an RNN is difficult due to vanishing/exploding gradients. [15]

Long Short Term Memory [15] addresses the problem of learning long-range dependencies by augmenting the RNN with a memory cell vector $c_t \in \mathbb{R}^n$ at each time step. Concretely, one step of an LSTM takes as input $x_{t,} h_{t-1}, c_{t-1}$ and produces h_t, c_t via the following intermediate calculations:

$$\begin{aligned} i_t &= \sigma\left(W^i x_t + U^i h_{t-1} + b^i\right) \\ f_t &= \sigma(W^f x_t + U^f h_{t-1} + b^f) \\ o_t &= \sigma(W^o x_t + U^o h_{t-1} + b^o) \\ g_t &= tanh(W^g x_t + U^g h_{t-1} + b^g) \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \qquad \ldots\ldots\ldots (5)$$

where,

$\sigma(.)$ *and* $\tanh(.)$ are element-wise sigmoid and hyperbolic tangent functions,

$\odot$ is the element-wise multiplication operator, and

i_t, f_t, o_t are referred to as input, forget, and output gates.

For recurrent neural network language model, let V be the fixed size vocabulary of words. A language model specifies a distribution over w_{t+1} given the historical sequence $w_{1:t} = [w_1, \dots, w_t]$. A recurrent neural network language model does this by applying an affine transformation to the hidden layer followed by a softmax:

$$\Pr(w_{t+1} = j | w_{1:t}) = \frac{\exp(h_t.p^j + q^j)}{\sum_{j' \in V} \exp(h_t.p^{j'} + q^{j'})} \dots\dots\dots\dots\dots\dots (6)$$

where,

p^j is the *j-th* column of $P \in R^{\mathrm{m\ x\ |V|}}$,

q^j is the bias term

Similarly for a conventional RNN-LM which usually takes words as inputs, if $w_t = k$, then the input to the RNN-LM at t is the input embedding x^k, the k-th column of the embedding matrix $X \in R^{\mathrm{n\ x\ |V|}}$. Here, we just replace the input embedding **X** with the output from a character-level convolutional neural network.

If we denote $w_{1:t} = [w_1, \dots, w_t]$ to be the sequence of words in the training corpus, training involves minimizing the negative log-likelihood (NLL) of the sequence.

$$NLL = -\sum_{t=1}^{T} logPr(w_t | w_{1:t-1}) \dots\dots\dots\dots\dots\dots\dots (7)$$

which is typically done by truncated backpropagation through time.

3.2.1.4 Stochastic Gradient Descent

Stochastic Gradient Descent follows the negative gradient descent of the objective after seeing only a single or a few training examples, instead of the full training set to compute the next update to parameters. SGD addresses the high cost of running back propagation over the full training set and yet lead to fast convergence.

The algorithm is given as

```
for i in range (m):
```

$$\theta_j = \theta_j - \alpha(y'^i - y^i)x_j^i \quad \ldots\ldots\ldots\ldots\ldots (8)$$

where,

y'^i is the actual i-th y-value

y^i is the i-th prediction

α is the learning rate

3.2.1.5 Perplexity

In language model, perplexity is mainly used to test the quality of the model. Perplexity gives a measure of how well a probability distribution or probability model predicts a sample. The quality of model is inversely proportional the value of perplexity.

The perplexity of a model over a sequence $[w_1, \ldots, w_T]$ is given by

$$P = \exp\left(\frac{NLL}{T}\right) \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (9)$$

Where,

NLL is Negative-log likelihood, and

T is sequence length

3.3 System Design

3.3.1 Flow Diagram

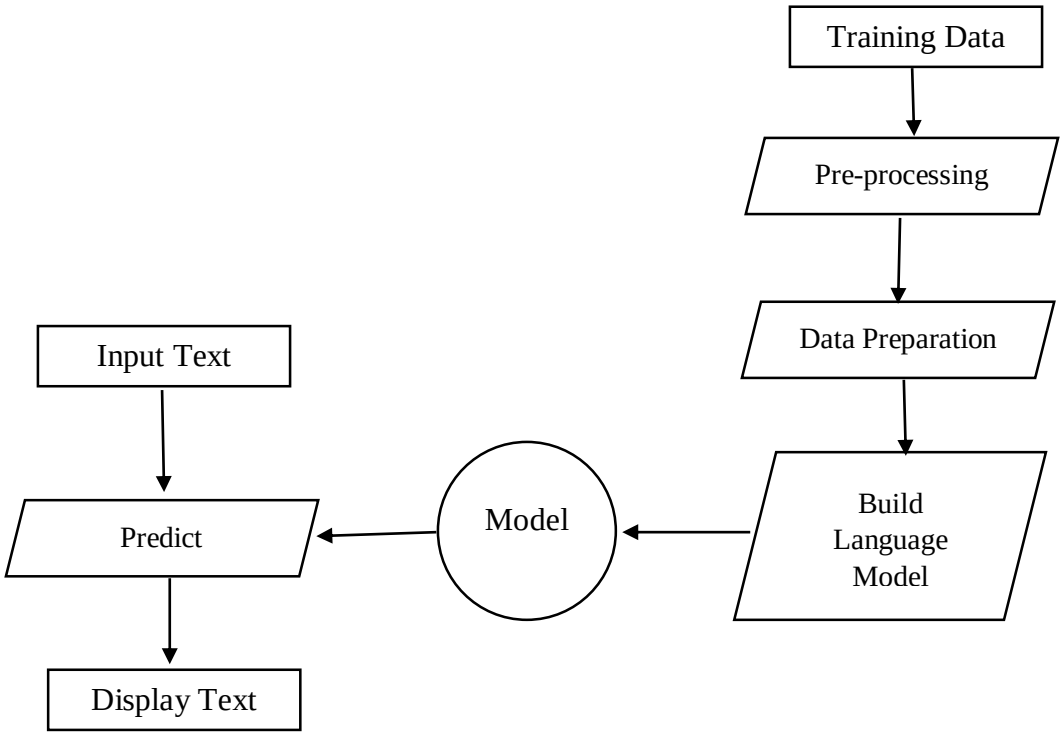

Figure 7 - Flow diagram of the system

Figure 7 shows the flow diagram of the system. First, the training data is acquired and then it is preprocessed. After preprocessing, the data is prepared to pass as input to the language model architecture. Then, the language model is built by training over the input data. This model is then stored in secondary memory and used for inference by deploying it into a system (here, web-based application).

3.3.2 Activity Diagram

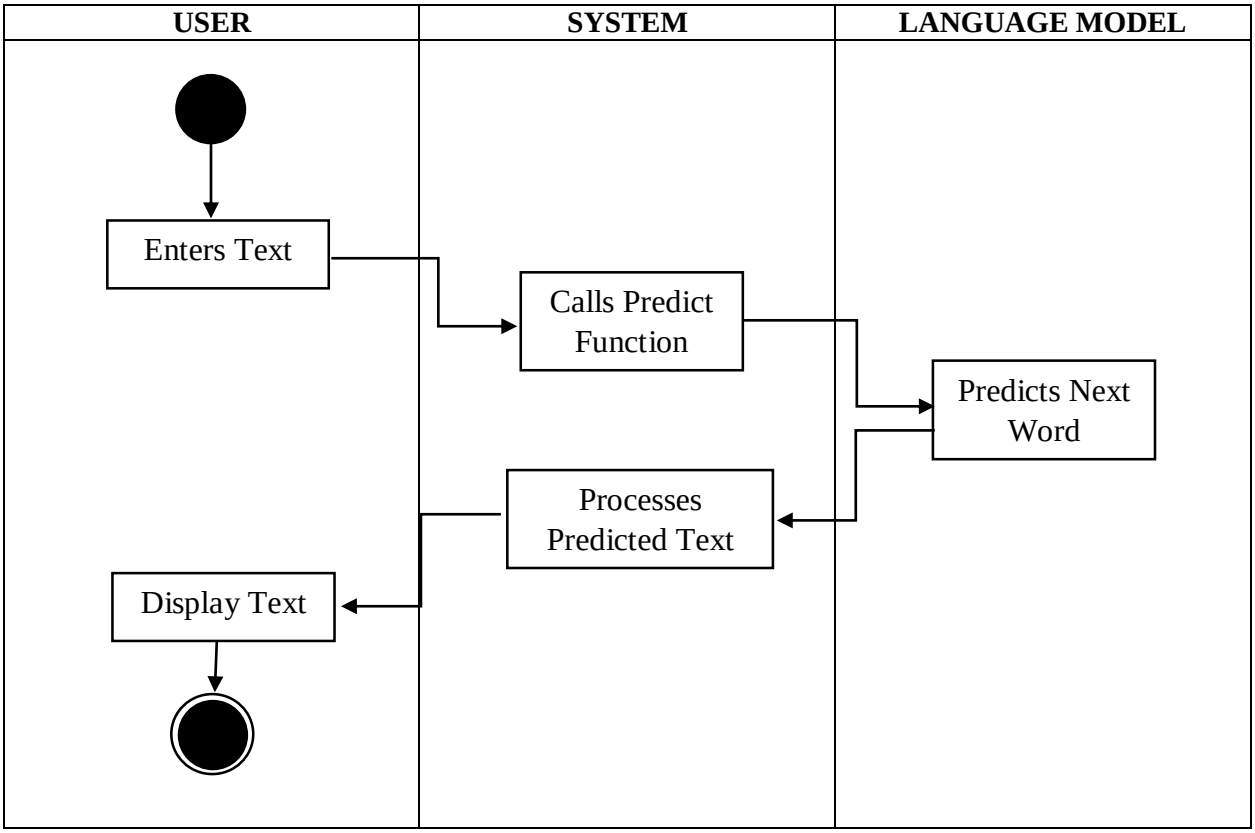

Figure 8 – Activity diagram of predicting next word

Figure 8 shows the activity diagram of predicting the next word by the system. Here, the user starts the process by entering some text (seed input) into the system. The system than calls the predict function on the trained language model by passing the seed input. The language model then assigns a probability distribution over the vocabulary set V and returns it to the system. The system now takes the index of maximum probability and extracts the corresponding word from the pre-built word dictionary. The extracted word is then appended to the seed input and displayed to the user.

CHAPTER 4: IMPLEMENTATION AND EVALUATION

4.1 Tools and Technologies Used

This section describes the tools and technologies used in the project.

CASE tools:

- Gliffy

Client Side:

- HTML is used structure the user interface and display the output results.
- Twitter Bootstrap Framework is used to beautifying the user interface.
- JavaScript/JQuery is used to make HTTP requests to the server by implementing AJAX

Server Side:

- Python programming language is used as the server-side programming language.
- Flask micro framework is used to implement a simple web user interface. It is used to start the web application and display the input field and the result.
- OS and NumPy are used for file handling and array processing functionalities respectively.
- Keras machine learning library is used to build the language model architecture and train the language model architecture.

Hardware:

- The language model was trained on Google Colaboratory having the following specs:
    a. GPU: 1 x Tesla K80, having 2496 CUDA cores, 12 GB GDDR5 VRAM
    b. CPU: 1 x single core hyper threaded Xeon processors @ 2.3 Ghz
    c. RAM: 1 x 12GB
- The web application was deployed in a Dell Vostro 3458 laptop having the following specs:

a. Ubuntu 18.04 LTS OS
b. CPU: Intel Core i5-5200U
c. RAM: 1 x 8GB DDR3

4.2 Implementation

4.2.1 Language Model Architecture

The language model architecture is implemented using Keras, a tensor-flow based machine learning library. The language model applied to an example sentence is shown in figure.

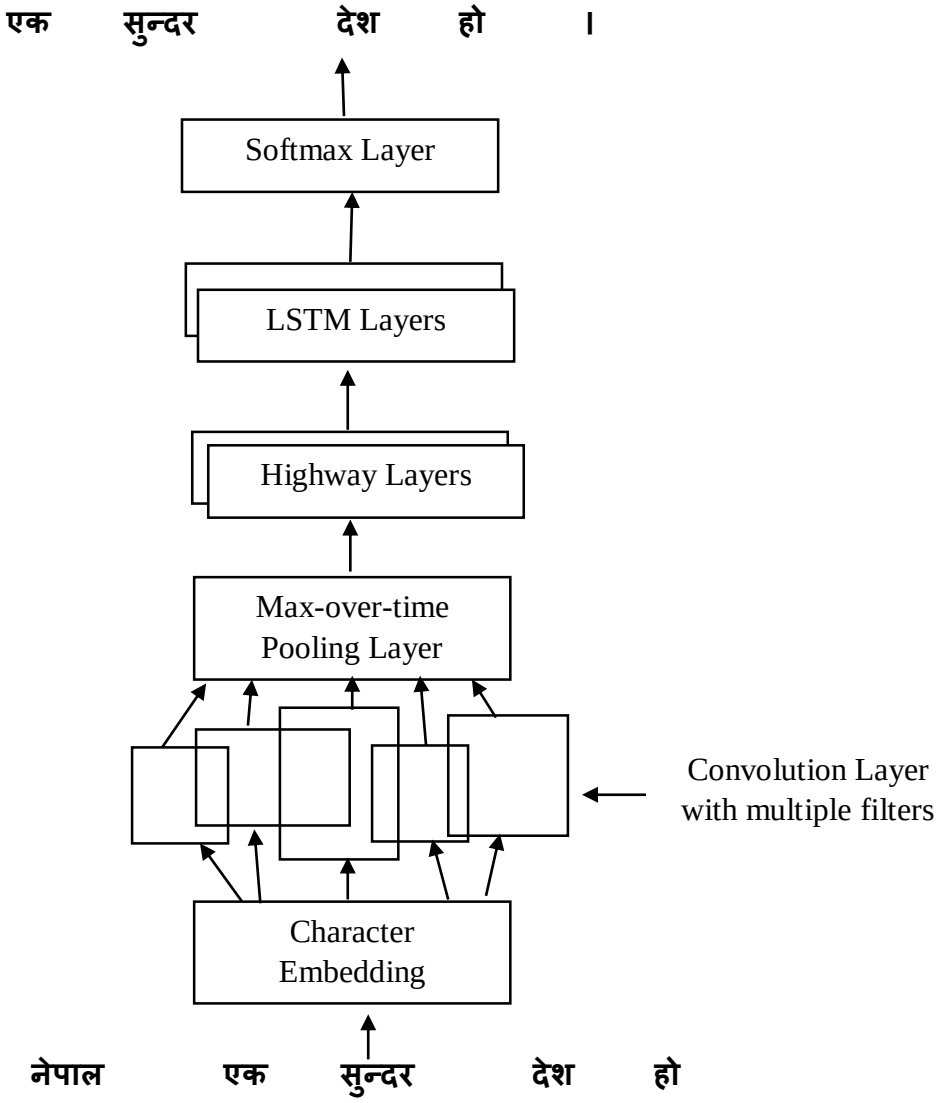

Figure 9 – Language model architecture on an example sentence

Figure 9 shows the language model architecture learning from an example sentence. Here, the token **'सुन्दर'** is fed into the language model architecture by representing it with a matrix. The matrix is formed by the concatenation of each character vector of the characters in the token. Then a number of filters are applied on the matrix and feature maps are produced which are max-pooled over time. Then, the output is run through two layers of Highway network and finally the two layers of LSTM learn the sequence. Finally, the softmax layer learns the probability distribution over a set of vocabulary. Here, it assigns maximum probability to the token **'देश'**.

4.2.3 Hyperparameters

The language model was trained by selecting the following hyperparameteres.

Table 5 – Hyperparameters Selected to Train Language Model

Hyperparameters	Value
Character Vector Size	15
Max Word Length	30
Vocabulary Size	60000
Feature Maps	[25, 50, 75, 100, 150, 200]
Kernels	[1, 2, 3, 4, 5, 6]
Learning Rate	1.0
Optimizer	SGD
RNN Size	300
Highway Layers	2
Batch Normalization Layer	1
LSTM Layers	2
Epochs	25

Hyperparameters	Value
Sequence Length	35
Dropout	0.5

4.2.4 Description of Major Files and Classes

File: dataPrepapre.py

This is a Python script file which reads the XML files from training corpus and concatenates the tokens based on the algorithm in figure. It converts morphologically-tokenized tokens into orthographic tokens. It reads raw XML files and transforms into orthographic tokens and then using the XML structure of the data, it is tokenized into paragraphs. The paragraphs are enclosed using XML <p> tag and written into a single large file.

File: createFontErrorDictionary.py

This Python script file creates a dictionary of tokens with font conversion errors and stores it in an XML file. The XML file is then manually updated with replacement tokens and used in the data preprocessing phase.

File: dataPreprocess.py

This Python script file performs all the data filtering explained in Section. The main functions of this file are follows:

1. removeNonDevanagraiTokens(paragraph)

   The input to this function is a paragraph. Then using regular expression, non-devnagari tokens are recognized and removed. The output of this function is a paragraph with no non-devanagari tokens.

2. replaceTokens(paragraph)

The input to this function is the output of the function removeNonDevanagraiTokens(). The tokens in the input paragraph are replaced if they exists in the dictionary created by createFontErrorDictionary.py. The output of this function is paragraph with tokens that have no font conversion errors.

3. removeParagaphs(paragraphs)

   The entire paragraph list is given as input to the function. The function then ignores the paragraphs with token length less than 35 and writes all other paragraphs into a single text file.

File: split.py

This Python script file uses scikit-learn Python modules' *train_test_split* function to split the corpus into train, validation and test corpus.

File: textToTensor.py

This Python script file converts the raw tokens into tensors suitable for fitting into the language model architecture. It mainly utilizes numPy Python library for its operations. The tensors are stored in '.npz' files. This python script file also draws statistics about the data (vocab size, character vocab size, maximum word length). This Python script file also builds the token dictionary necessary for inference during deployment of the project.

Class: Model

The class model implements the language model architecture in figure. It consists of a train function which is called to train the model. It uses Keras library to build the architecture.

File: train.py

This Python script file calls the train function of the class Model. It first loads all the tensors from the '.npz' files into the memory. Then it draws out 35 tokens (sequence length) with a batch size

of 20 from the tensor tokens and feeds it into the language model architecture during training process. The file is run on Google Colab for 25 epochs with each epoch running for an average of 845 seconds. At each time step, it calculates the perplexity on the validation corpus and save the trained model weights in '.h5' file.

File: evaluate.py

This Python script file loads the model weights obtained from the last epoch of the training process and uses it to evaluate the perplexity of the language model on the test corpus. In the evaluation process a batch size of 1 and sequence length of 1 is used. All the other hyperparameters are same as used in the training process.

File: app.py

This Python script file starts the web-application. The file first loads the model weights and makes it ready for prediction. It handles requests and response between the user and the system.

4.3 Testing

In order to test the model the corpus was divided in the ratio of 4:1 (Train: Test) after preprocessing. The language model was trained on the train corpus and the weights of the model were saved in secondary storage. The model was then evaluated on the test corpus. The same hyperparamters in Table 5 were used to evaluate the model, except for the sequence length and batch size which were set to one. The model achieved a perplexity score of 378.81 on the test corpus i.e. in each prediction the language model is equally likely to predict 379 words as the correct one.

Perplexity is an indication of how confused the language model is. A lower perplexity score is considered to be better than a higher perplexity score.

CHAPTER 5: CONCLUSIONS AND LIMITATIONS

5.1. Conclusion

Language modeling is an important task in the field of NLP. Language modeling is useful for many downstream tasks in NLP such as Machine Translation, Speech Recognition, POS Tagging etc. The introduction of word embeddings and the advent of deep learning saw increase in language modeling research. However, the research works only considered English and other European languages and a language model architecture developed for one language does not necessarily work for other languages. Some of the reasons for this is variation in language characteristics, vocab size, etc. For example, the language model architecture using the word vectors do not capture the sub-word information which is a key part of morphologically rich languages such as Nepali. Hence, an NLM using character-level CNN and LSTM was developed to train a language model for Nepali language. However, the language model did not perform well on testing. Hence, further analysis is required to determine a concrete conclusion.

5.2. Limitations

The system is limited by the model it is using. The language model which could only achieved a perplexity score of 378.81 on the test corpus. The limitations of the system are as follows:

- The language model only correctly predicts verbs and end-of-sentence punctuations.
- The system is only as good as the language model it is using.
- The system can only predict one word at time.
- The language model requires huge computing resources to train.

REFERENCES

[1]. Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C., "A Neural Probabilistic Language Model" Journal of Machine Learning Research, vol. 3, Feb 2003. [Online]. Available: http://www.jmlr.org/papers/volume3 /bengio03a/bengio03a.pdf [Accessed May. 4, 2019].

[2]. T. Mikolov, K. Chen, G. Corrado, J. Dean, "Efficient Estimation of Word Representations in Vector Space", Proc. Workshop at ICLR, 2013

[3]. Kim, Y., Jernite, Y., Sontag, D., & Rush, A. M., "Character-Aware Neural Language Model," in AAAI: Proceedings of the 13th AAAI Conference on Artificial Intelligence (AAAI-16), AAAI 2016, Phoenix, AR, USA, February 12-17, 2016.

[4]. Schwenk, Holger, Rousseau, Anthony, and Attik, Mohammed, "Large, pruned or continuous space language models on a GPU for statistical machine translation.," in Proceedings of the NAACL-HLT 2012 Workshop: Will We Ever Really Replace the N-gram Model? On the Future of Language Modeling for HLT, pp. 11–19. Association for Computational Linguistics, 2012

[5]. Dos Santos, C. N., and Zadrozny, B. "Learning Character-level Representations for Part-of-Speech Tagging," in Proceedings of ICML 2014

[6]. Mikolov, Tomas, Karafiat, Martin, Burget, Lukas, Cernocky, Jan, and Khudanpur, Sanjeev. "Recurrent neural network based language model," in INTERSPEECH, volume 2, pp. 3, 2010

[7]. Chen, S., and Goodman, J. "An Empirical Study of Smoothing Techniques for Language Modeling" Technical Report, Harvard University

[8]. Mikolov, T.; Deoras, A.; Kombrink, S.; Burget, L.; and Cernocky, J. 2011. Empirical Evaluation and Combination of Advanced Language Modeling Techniques. In Proceedings of INTERSPEECH.

[9]."Nepali Monolingual written corpus, ELRA catalogue (http://catalog.elra.info), ISLRN: 325-796-965-405-9, ELRA ID: ELRA-W0076"

[10]. Hardie A., Lohani R., Regmi B., & Yadava Y., "Categorization for automated morphosyntactic analysis of Nepali: introducing the Nelralec Tagset", July 2005, Nelralec/Bhasha Sanchar Working Paper 2

[11]. K. Parajuli, "पदवर्ग," in Ramro Rachana Mitho Nepali, Sahayogi Press, 1966

[12]. Curriculum Development Center (CDC), “यात्रा सुरु रौं”in Nepali Grade 9, CDC, 2009

[13]. LeCun Y., Bottou L., Bengio Y., Haffner P., “Gradient-based learning applied to document recognition”, in proceedings of the IEEE 86 (11), 2278-2324, 1998

[14]. Srivastava R. K., Greff K., and Schmidhuber J. “Training Very Deep Networks”, in proceedings NIPS, 2015

[15]. Sundermeyer, M., Schluter R., and Ney H., “LSTM Neural Networks for Language Modeling,” in proceedings of INTERSPEECH, 2012

APPENDIX I

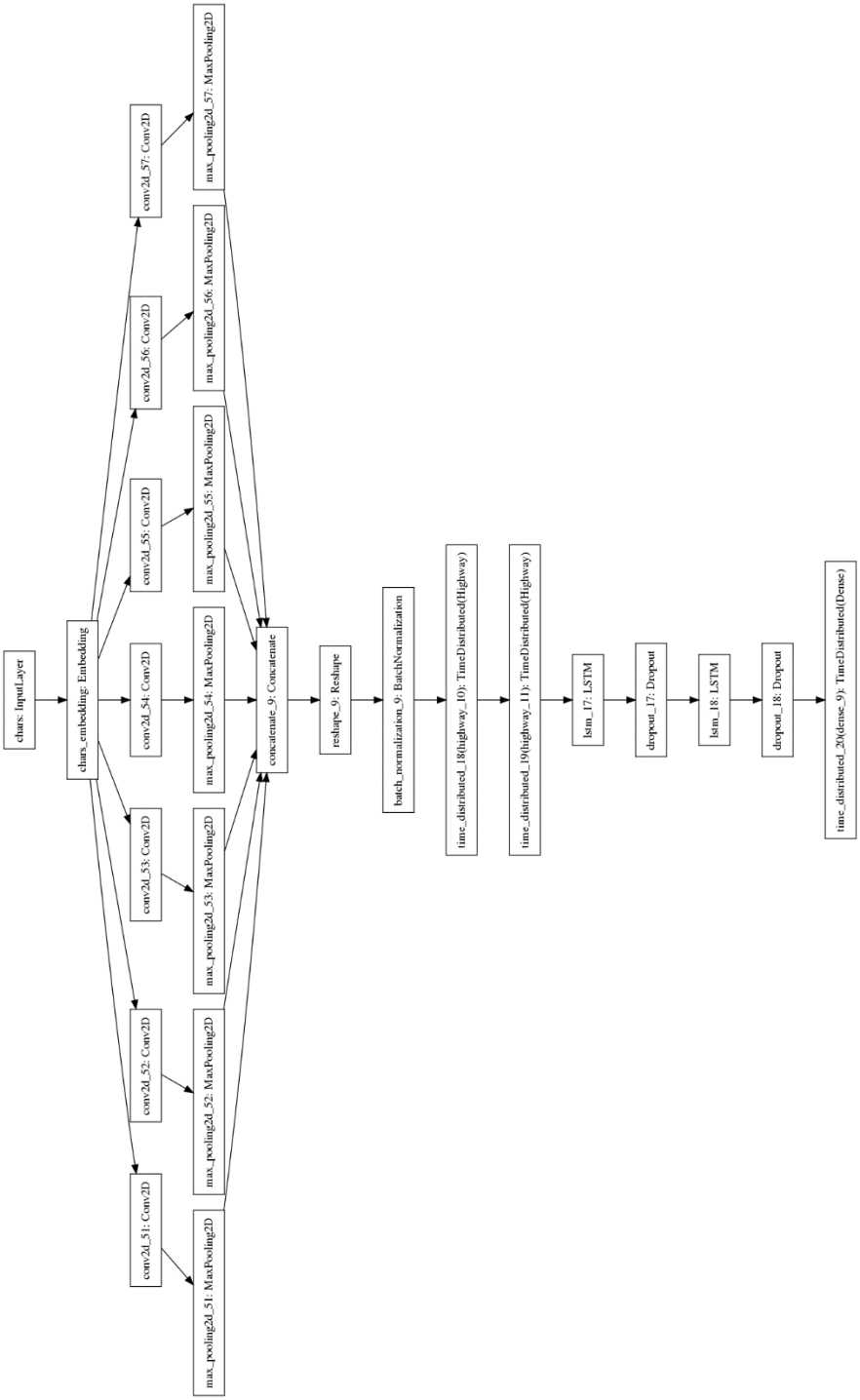

Figure 10 – Language model architecture in Keras

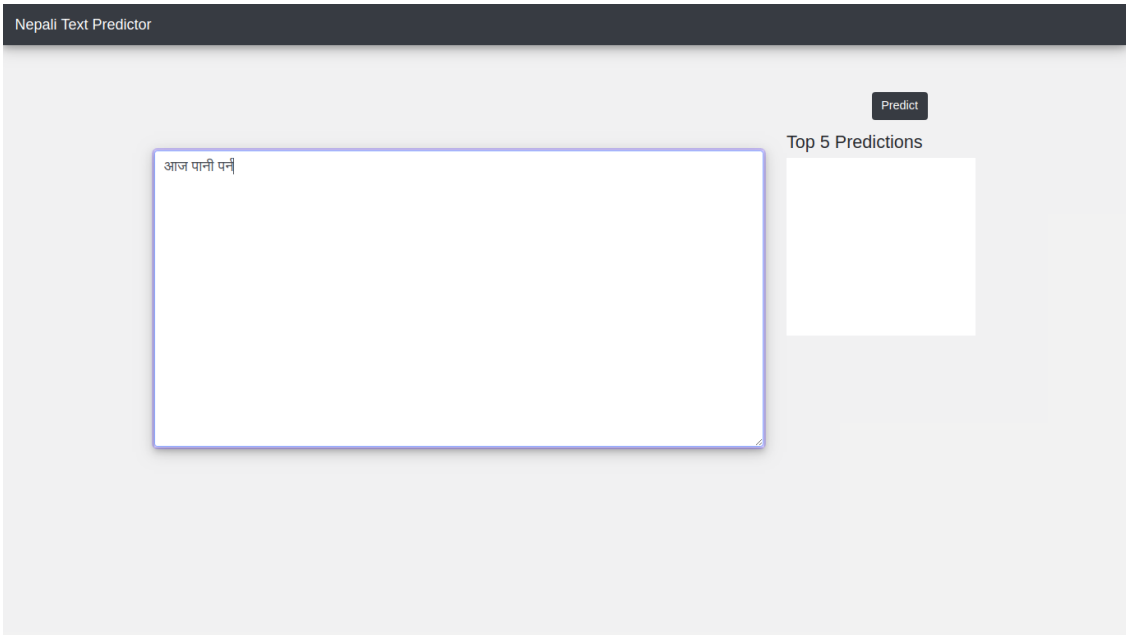

Figure 11 – User interface with an example seed input

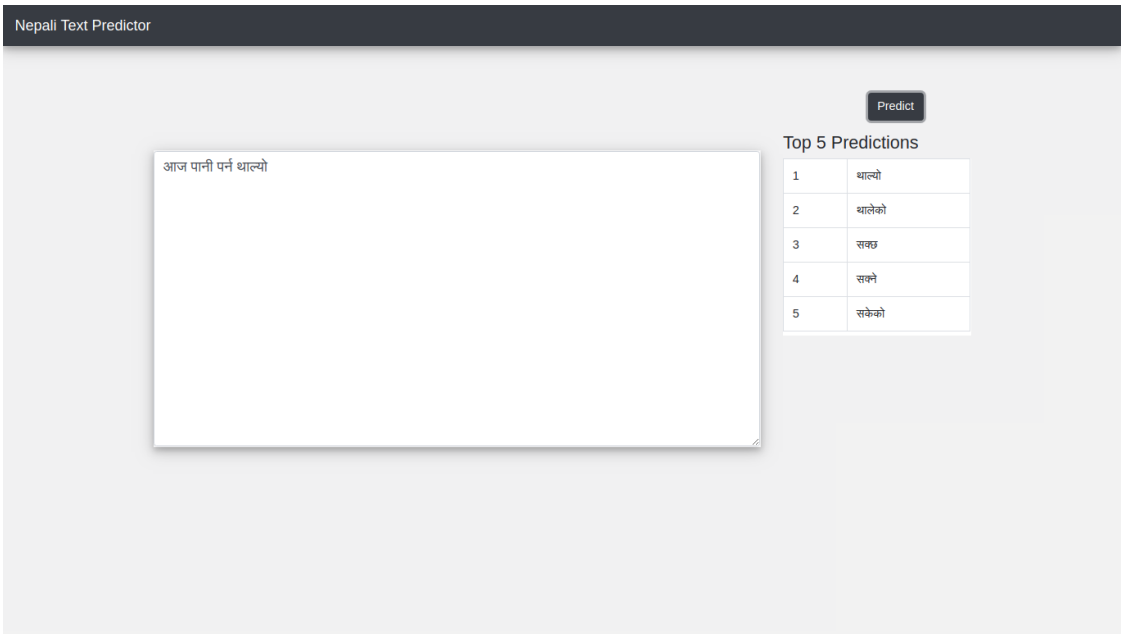

Figure 12 – Output result displayed by the system